Import Library

In [1]:
```
import pandas as pd
import numpy as np
import lightgbm as lgb
from sklearn.model_selection import KFold
from sklearn.metrics import mean_absolute_error
import seaborn as sns
import matplotlib.pyplot as plt
```

Load Data

In [2]:
```
df = pd.read_csv("/kaggle/input/competitions/cpe-232-insurance-premium-prediction/train.csv")
```

In [3]:
```
test_df = pd.read_csv("/kaggle/input/competitions/cpe-232-insurance-premium-prediction/test.csv")
```

Clean Data & EDA

In [4]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 520000 entries, 0 to 519999
Data columns (total 20 columns):
 #   Column                Non-Null Count   Dtype  
---  ------                --------------   -----  
 0   id                    520000 non-null  int64  
 1   Age                   460257 non-null  float64
 2   Gender                489711 non-null  object 
 3   Annual Income         366970 non-null  float64
 4   Marital Status        386147 non-null  object 
 5   Number of Dependents  458033 non-null  float64
 6   Education Level       436067 non-null  object 
 7   Occupation            312415 non-null  object 
 8   Health Score          427827 non-null  float64
 9   Location              396667 non-null  object 
 10  Policy Type           459628 non-null  object 
 11  Previous Claims       491570 non-null  float64
 12  Vehicle Age           440458 non-null  float64
 13  Credit Score          436376 non-null  float64
 14  Insurance Duration    441736 non-null  float64
 15  Customer Feedback     430946 non-null  object 
 16  Smoking Status        419824 non-null  object 
 17  Exercise Frequency    451319 non-null  object 
 18  Property Type         439850 non-null  object 
 19  Premium Amount        520000 non-null  float64
dtypes: float64(9), int64(1), object(10)
memory usage: 79.3+ MB
```

In [5]:
```
df.isnull().sum()
```

Out[5]:
```
id                         0
Age                    59743
Gender                 30289
Annual Income         153030
Marital Status        133853
Number of Dependents   61967
Education Level        83933
Occupation            207585
Health Score           92173
Location              123333
Policy Type            60372
Previous Claims        28430
Vehicle Age            79542
Credit Score           83624
Insurance Duration     78264
Customer Feedback      89054
Smoking Status        100176
Exercise Frequency     68681
Property Type          80150
Premium Amount             0
dtype: int64
```

Every attribute has null.

In [6]:
```
df["Premium Amount"].describe()
```

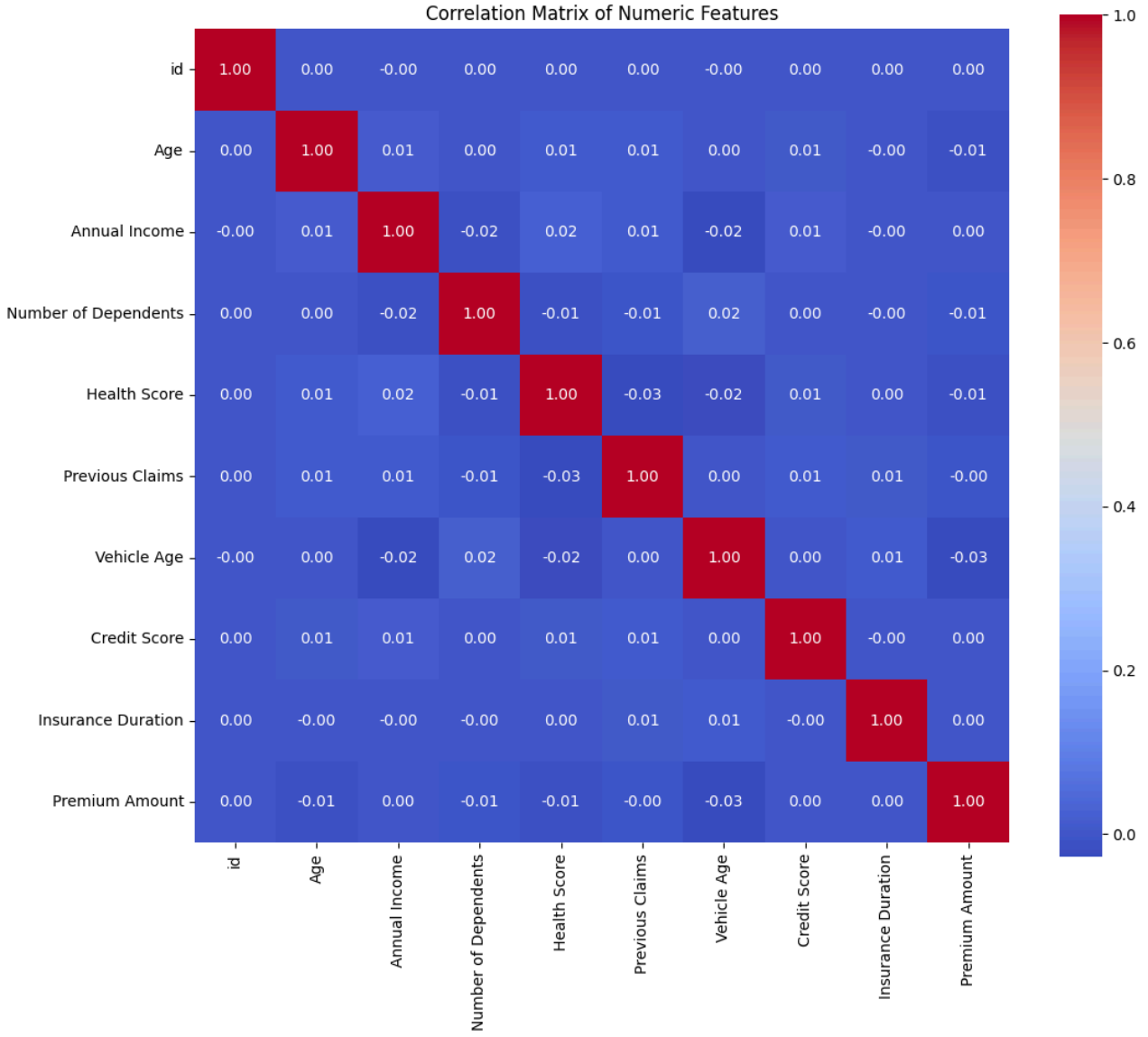
Out[6]:
```
count    520000.000000
mean        846.442148
std         822.968422
min           0.000000
25%         269.830000
50%         524.275000
75%        1217.242500
max        4413.150000
Name: Premium Amount, dtype: float64
```

In [7]:
```
numeric_df = df.select_dtypes(include=[np.number])

corr = numeric_df.corr()

plt.figure(figsize=(12, 10))
sns.heatmap(corr, annot=True, fmt=".2f", cmap='coolwarm', square=True)
plt.title('Correlation Matrix of Numeric Features')
plt.show()

print(corr['Premium Amount'].sort_values(ascending=False))
```

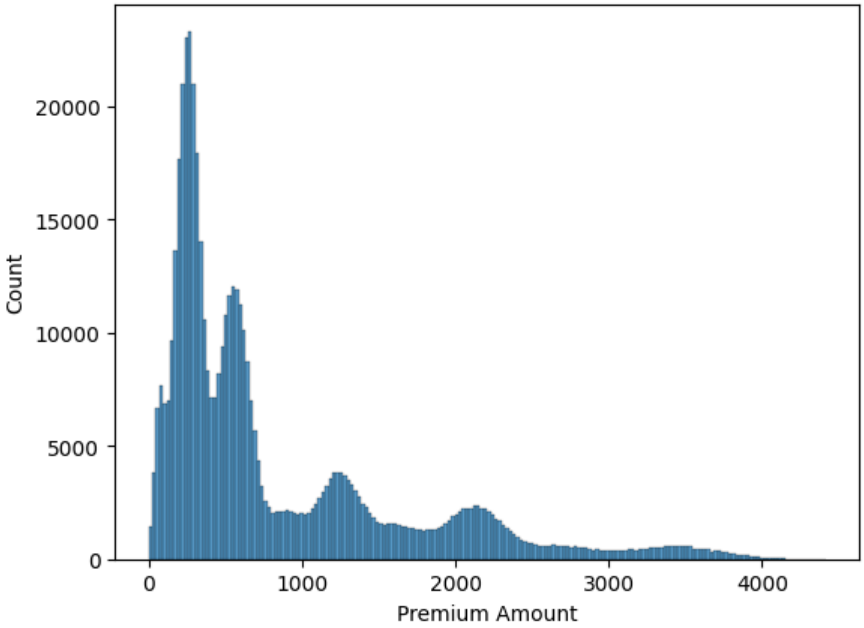



```
Premium Amount          1.000000
Insurance Duration      0.003744
Annual Income           0.001758
id                      0.001402
Credit Score            0.000095
Previous Claims        -0.005000
Number of Dependents   -0.005127
Age                    -0.012171
Health Score           -0.012503
Vehicle Age            -0.025424
Name: Premium Amount, dtype: float64
```

Max correlation 0.003 so it is not Linear regression.

In [8]:
```
sns.histplot(df['Premium Amount'])
```

Out[8]:
```
<Axes: xlabel='Premium Amount', ylabel='Count'>
```



right skewed frequency -> Log transform.

In [9]:
```
X = df.drop(columns=['id', 'Premium Amount']).copy()
y = df['Premium Amount'].copy()
X_test = test_df.drop(columns=['id']).copy()
```

In [10]:
```
num_cols = X.select_dtypes(include=[np.number]).columns.tolist()
cat_cols = X.select_dtypes(include=[object]).columns.tolist()

for col in num_cols:
    fill_val = X[col].median()
    X[col] = X[col].fillna(fill_val)
    X_test[col] = X_test[col].fillna(fill_val)

for data in [X, X_test]:
    data['Income_Age_Ratio'] = data['Annual Income'] / (data['Age'] + 1)
    data['Health_Credit'] = data['Health Score'] * data['Credit Score']

for col in cat_cols:
    X[col] = X[col].astype('category')
    X_test[col] = X_test[col].astype('category')
```

Impute Numerical columns with median and Filled Na category columns with UNKNOWN.

In [11]:
```
y_log = np.log1p(y)
```

Label log transform.

Model Training

In [14]:
```
params = {
    'objective': 'regression',
    'metric': 'mae',
    'boosting_type': 'gbdt',
    'num_leaves': 127,
    'learning_rate': 0.01,
    'feature_fraction': 0.75,
    'bagging_fraction': 0.75,
    'bagging_freq': 5,
    'device': 'gpu',
    'random_state': 42,
    'verbosity': -1
}
```

In [17]:
```
import warnings
warnings.filterwarnings('ignore')

kf = KFold(n_splits=5, shuffle=True, random_state=42)
oof_preds = np.zeros(len(X))
test_preds = np.zeros(len(X_test))

for fold, (train_idx, val_idx) in enumerate(kf.split(X, y_log)):
    X_train, X_val = X.iloc[train_idx], X.iloc[val_idx]
    y_train_log, y_val_log = y_log.iloc[train_idx], y_log.iloc[val_idx]

    train_set = lgb.Dataset(X_train, label=y_train_log, categorical_feature=cat_cols)
    val_set = lgb.Dataset(X_val, label=y_val_log, reference=train_set, categorical_feature=cat_cols)

    model = lgb.train(
        params,
        train_set,
        num_boost_round=15000,
        valid_sets=[train_set, val_set],
        valid_names=['train', 'valid'],
        callbacks=[
            lgb.early_stopping(stopping_rounds=300),
            lgb.log_evaluation(period=500)
        ]
    )

    fold_preds_log = model.predict(X_val)
    oof_preds[val_idx] = np.expm1(fold_preds_log)

    test_preds += np.expm1(model.predict(X_test)) / kf.n_splits

    print(f"Fold {fold+1} Completed.")
```

```
Training until validation scores don't improve for 300 rounds
[500]   train's l1: 0.780225    valid's l1: 0.788587
[1000]  train's l1: 0.771843    valid's l1: 0.788234
[1500]  train's l1: 0.76459     valid's l1: 0.788286
Early stopping, best iteration is:
[1276]  train's l1: 0.767739    valid's l1: 0.788193
Fold 1 Completed.
Training until validation scores don't improve for 300 rounds
[500]   train's l1: 0.780005    valid's l1: 0.788997
[1000]  train's l1: 0.771596    valid's l1: 0.788661
Early stopping, best iteration is:
[1133]  train's l1: 0.769559    valid's l1: 0.788638
Fold 2 Completed.
Training until validation scores don't improve for 300 rounds
[500]   train's l1: 0.779413    valid's l1: 0.791476
[1000]  train's l1: 0.771057    valid's l1: 0.791082
Early stopping, best iteration is:
[1139]  train's l1: 0.768936    valid's l1: 0.791057
Fold 3 Completed.
Training until validation scores don't improve for 300 rounds
[500]   train's l1: 0.779991    valid's l1: 0.789043
[1000]  train's l1: 0.771619    valid's l1: 0.788665
[1500]  train's l1: 0.764312    valid's l1: 0.78863
Early stopping, best iteration is:
[1438]  train's l1: 0.765178    valid's l1: 0.788608
Fold 4 Completed.
Training until validation scores don't improve for 300 rounds
[500]   train's l1: 0.780661    valid's l1: 0.78654
[1000]  train's l1: 0.772348    valid's l1: 0.786114
Early stopping, best iteration is:
[1155]  train's l1: 0.770039    valid's l1: 0.7861
Fold 5 Completed.
```

Model Evaluation & Create Submission

In [19]:
```
overall_mae = mean_absolute_error(y, oof_preds)
print(f"Overall OOF MAE: {overall_mae:.4f}")

submission = pd.DataFrame({
    'id': test_df['id'],
```

```
    'Premium Amount': test_preds
})
submission.to_csv('sub_lgbm_kfold.csv', index=False)
print("Create LightGBM submission")
```

```
Overall OOF MAE: 565.1548
Create LightGBM submission
```

In [ ]: